Processes and CPU Scheduling

Today's Syllabus

- Process Concept
- Program vs Process
- Process in Memory
- Process States
- Process State Diagram
- Process Control Block (PCB)
- Context Switching

Processes and CPU Scheduling

Process Concept

Topics

- What is a Process?
- Program vs Process
- Process States
- PCB
- Context Switching

“When you double-click Chrome, what exactly happens inside the computer?”

“The Chrome application stored on your SSD is a program. When you execute it, the OS creates a process.”

Connection with Chapter 1

In Chapter 1, we learned:

Operating System = Resource Manager

The OS manages:

- CPU
- Memory
- Files
- I/O devices

But…

Who actually uses the CPU?

-> Processes

Therefore:

Process is one of the most fundamental concepts in an Operating System.

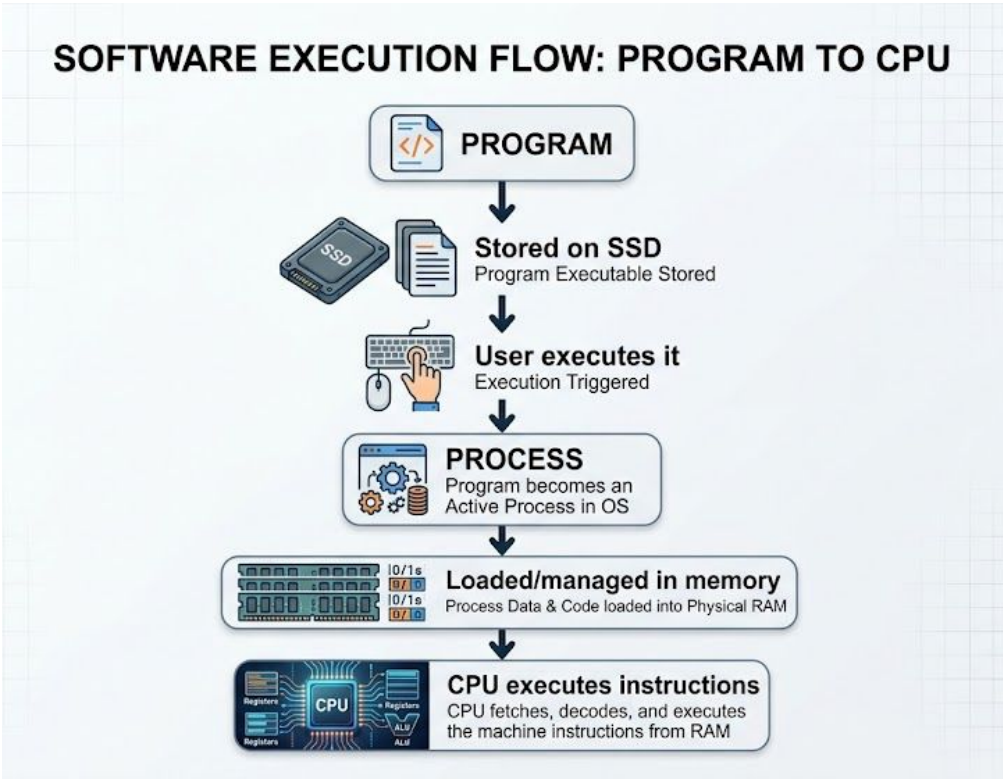
What is a Process?

Definition

A process is a program in execution.

A process is an **active entity**, whereas a program is a **passive entity**.

Example



Important

Program ≠ Process

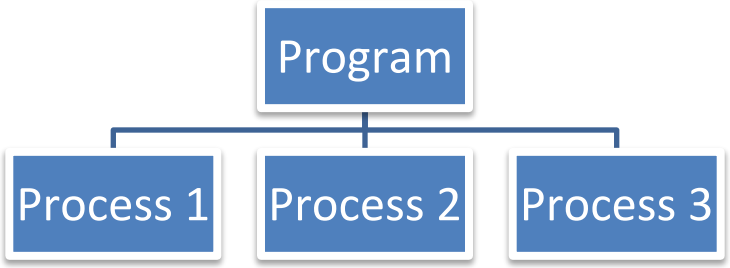
Program vs Process

Program	Process
Passive	Active
Stored on storage	Execution state maintained in memory/resources
Set of instructions	Program + current execution state/resources
Does not execute by itself	Executes
Can exist without running	Exists while execution is being managed

Example
Chrome.exe stored on disk → **Program**
Running Chrome → **Process**

Can One Program Have Multiple Processes?

Yes!
Suppose you launch the same application multiple times.



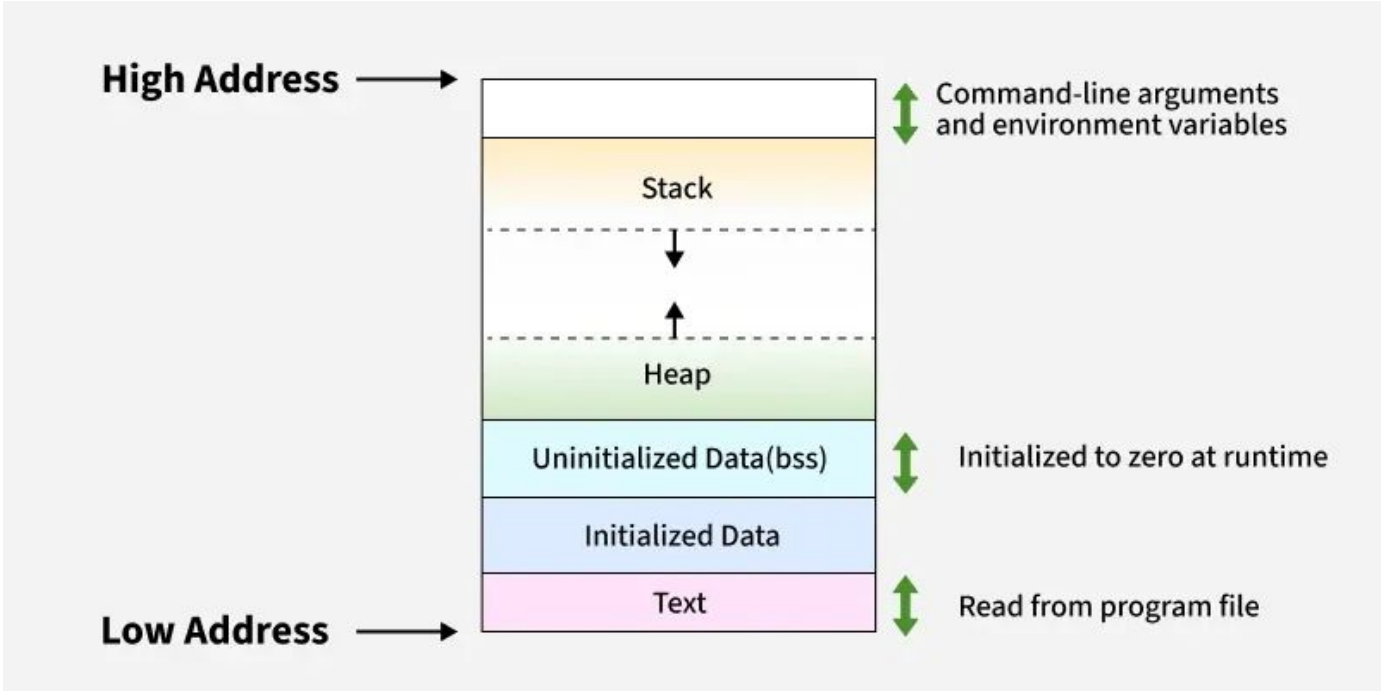
Each process has its own execution state and allocated resources.
Important Concept
One program can result in multiple processes.

What Does a Process Contain?

A process needs more than just program instructions.

Main components

- Program code
- Data
- Stack
- Heap
- CPU registers
- Program Counter
- Process state
- Other OS-managed resources


High Address
Command-line arguments
and environment variables
Stack
Heap
Uninitialized Data(bss)
Initialized to zero at runtime
Initialized Data
Text
Read from program file
Low Address

Process Memory Sections

1. Text / Code
Contains executable instructions.
2. Data
Contains global and static variables.
3. Heap
Used for dynamically allocated memory.
4. Stack
Used for:

- Function calls
- Local variables
- Function parameters
- Return addresses

Process State

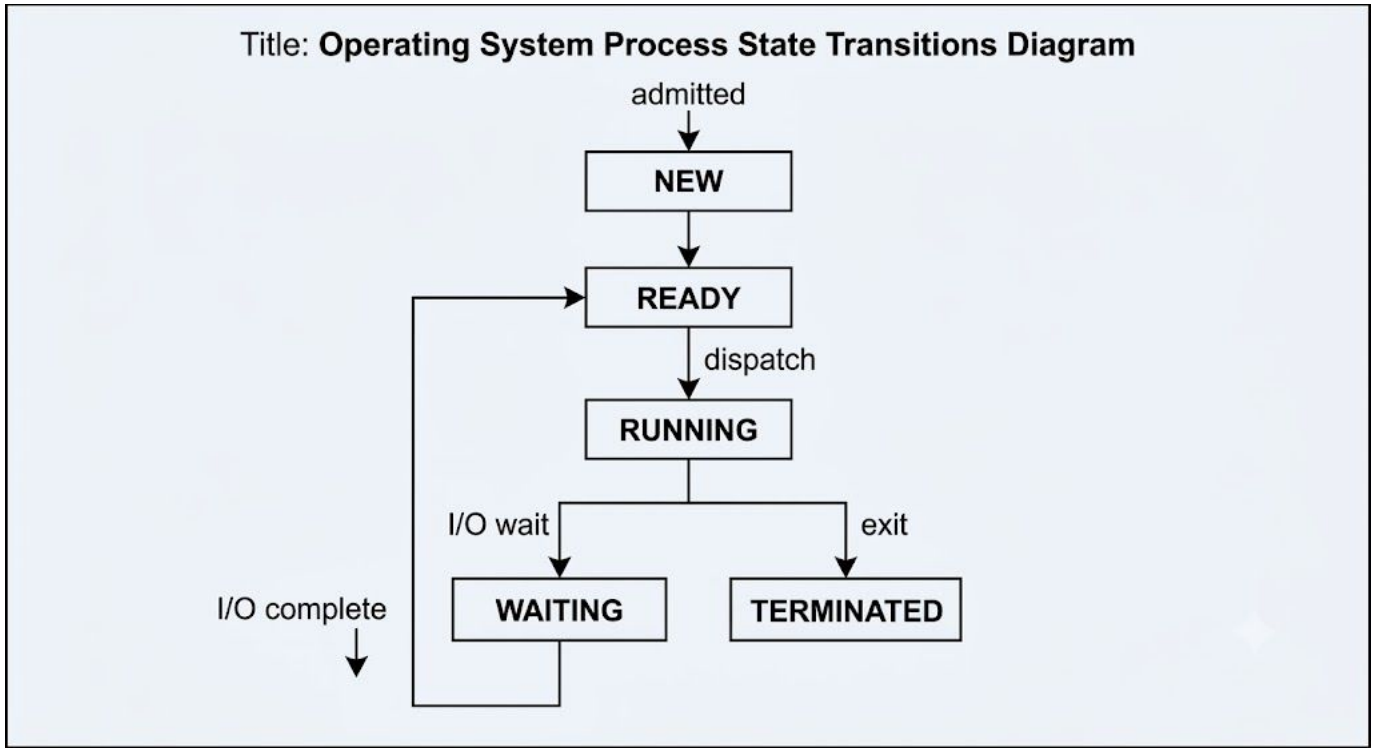
A process does not continuously execute on the CPU.

At different times, it can be in different states.

Basic States

1. New
2. Ready
3. Running
4. Waiting/Blocked
5. Terminated

Process State Diagram



New State

NEW
A process is in the **New** state when it is being created.
Example
You double-click:
VS Code
The OS begins creating a process.

User launches program
↓
Process creation
↓
NEW

Ready State

READY
The process is:
Ready to execute but waiting for the CPU.
Example
Suppose:

P1 → Running
P2 → Ready
P3 → Ready

P2 and P3 are ready but cannot execute because the CPU is currently occupied.
Important
Ready ≠ Running

Running State

RUNNING
The process is currently executing instructions on the CPU.

Ready
↓
CPU assigned
↓
Running

Important
On a single-core CPU:
At a given instant, only one process can actually execute on the CPU core.

Waiting / Blocked State

WAITING

A process enters the waiting state when it must wait for some event.

Examples

- Waiting for disk I/O
- Waiting for keyboard input
- Waiting for a signal
- Waiting for another process

Example

Process
↓
Requests disk read
↓
WAITING
↓
I/O completed
↓
READY

Terminated State

TERMINATED
The process has finished execution or has been terminated.
Example

Running
↓
exit()
↓
Terminated

The OS can then reclaim resources associated with the process.

Process State Transitions

Important Transitions

Transition	Meaning
New → Ready	Process admitted
Ready → Running	CPU assigned
Running → Ready	Preempted
Running → Waiting	Waiting for event/I/O
Waiting → Ready	Event/I/O completed
Running → Terminated	Execution completed

Why Does Running → Ready Happen?

Suppose:
P1 is Running

But its CPU time is taken away by the scheduler.
Then:

P1
↓
Running
↓
Preemption
↓
Ready

Process Control Block

PCB
Definition
Process Control Block (PCB) is a data structure maintained by the Operating System containing information about a process.
Think of it as:
“The OS’s record/card for a process.”

What Does a PCB Contain?

Typical PCB Information

1. Process ID
2. Process State
3. Program Counter
4. CPU Registers
5. CPU Scheduling Information
6. Memory Management Information
7. Accounting Information
8. I/O Status Information

PCB Diagram

PROCESS CONTROL BLOCK
Process ID (PID)
Process State
Program Counter
CPU Registers
Scheduling Information
Memory Information
Accounting Information
I/O Status Information

“If the OS needs to stop a process temporarily and resume it later, it needs to remember exactly where that process was.
The PCB stores important execution information.”

Program Counter

Program Counter (PC)
The Program Counter stores the address of the **next instruction to be executed**.
Example
Suppose:
Instruction 100
Instruction 104
Instruction 108
Instruction 112
If the process is currently at instruction 108:
PC → 112

The OS needs this information to resume execution correctly.

CPU Registers

Registers contain temporary information required during execution.
Examples include:

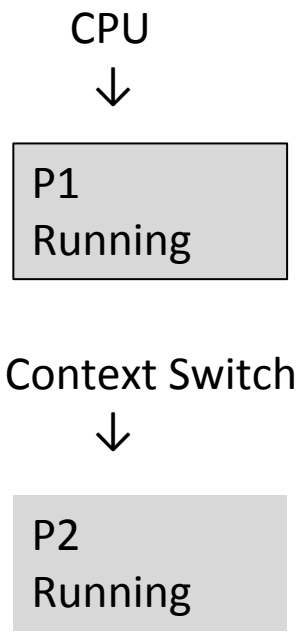
- General-purpose registers
- Stack pointer
- Status/flags registers
- Other architecture-specific registers

Why PCB stores them?
When a process stops running, its relevant CPU state must be saved so that execution can later continue correctly.

Context Switching

What is Context Switching?
Context switching is the process of saving the state of a running process and restoring the saved state of another process.
Example
P1 Running
↓
Save P1 Context
↓
Load P2 Context
↓
P2 Running

Context Switch Diagram



Behind the scenes

P1 registers → saved
P1 PC → saved

↓

P2 registers → restored
P2 PC → restored

Why is Context Switching Necessary?

Because multiple processes need CPU time.
Example:
P1 → P2 → P3 → P1 → P2

The OS scheduler decides when processes run.
Important
Context switching provides the mechanism for switching execution between processes.

Context Switch Overhead

Context switching requires CPU time.
During the switch:

- Current state is saved
- New state is restored
- CPU starts executing the new process

Important
Context switching is overhead because the CPU is performing switching work rather than executing user-process instructions.

Question
Does context switching increase useful application work?
➡ **No.**

Mini Problem 1: Process States

Question

A process is executing. It requests data from a disk.

What state transition occurs?

A. Ready → Running

B. Running → Waiting

C. Waiting → Running

D. Running → New

Mini Problem 1: ANSWER

Question

A process is executing. It requests data from a disk.

What state transition occurs?

A. Ready → Running

B. Running → Waiting

C. Waiting → Running

D. Running → New

Answer

B. Running → Waiting

Because the process must wait for I/O completion.

Mini Problem 2: State Transition

A process is waiting for disk I/O. The disk operation completes.
What happens next?
Waiting
↓
?

Answer
Waiting → Ready
Important
It does **not necessarily go directly to Running**.
Why?
Because another process may currently be using the CPU.

Mini Problem 3: Context Switching

Suppose:
P1 = Running
P2 = Ready
The scheduler decides to run P2.
What happens?

Mini Problem 3: Context Switching ANSWER

Suppose:
P1 = Running
P2 = Ready
The scheduler decides to run P2.
What happens?

Answer

P1
↓
Save context
↓
PCB(P1)
↓
Load context of P2
↓
P2 Running

Key Concepts

- Context of P1 is saved
- Context of P2 is restored
- CPU starts/resumes P2

Numerical Foundation

For every process we may know:

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	4

Arrival Time (AT)
Time at which the process enters the ready system/queue.

Burst Time (BT)
Amount of CPU execution time required by the process in the simplified scheduling model.

Simple Timeline

Time → 0 5 8 12
| P1 | P2 | P3 |

- P1 executes for 5 units
- P2 executes for 3 units
- P3 executes for 4 units

Process	CT
P1	5
P2	8
P3	12

Important Terms

Process
Program in execution.
Process State
Current status of the process.
PCB
OS data structure containing process information.
Context Switch
Switching CPU execution from one process to another.
Ready Queue
Queue/list of processes ready to execute.
CPU Burst
Period during which a process executes on the CPU.

MCQs

Q1. Which statement correctly defines a process?

A. A file stored on disk B. A program in execution

C. A CPU instruction D. A memory block

Q2. Which state means that a process is waiting for CPU allocation?

A. Running B. Terminated C. Ready D. Waiting

Q3. Which data structure contains information about a process?

A. FAT B. PCB C. MBR D. Cache

Q4. Which information indicates the next instruction to be executed?

A. PID B. Program Counter C. File descriptor D. Stack frame

Q5. What happens during a context switch?

A. A program is permanently deleted B. CPU is turned off

C. State of one process is saved and another is restored D. RAM is formatted

B

C

B

B

C

Summary

Process Concept

We learned:

- Process = Program in execution
- Program vs Process
- Process memory layout
- Process states
- State transitions
- PCB
- Program Counter
- CPU Registers
- Context Switching
- Arrival Time
- CPU Burst Time